

Argo CD Installation on Kubernetes

Practical step-by-step guide for a self-managed Rocky Linux Kubernetes homelab. Example control-plane IP: **192.168.178.49**. Run kubectl commands from a machine with cluster-admin access.

1. Prerequisites

```
kubectl get nodes -o wide
kubectl cluster-info
kubectl get pods -n kube-system
```

Confirm your control-plane and at least one worker are Ready, CoreDNS is healthy, and the cluster can download images from the internet. Investigate NotReady nodes before relying on them for workloads.

2. Create namespace and install

```
kubectl create namespace argocd
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

The stable branch is convenient for a lab, but it can change over time. For reproducible installations, pin a tested Argo CD release manifest.

3. Verify Argo CD components

```
kubectl get pods -n argocd -w
kubectl get deployments,statefulsets -n argocd
kubectl get svc -n argocd
```

Wait until all required Pods are Running and Ready. The argocd-server Service is normally ClusterIP by default.

4. Access the web interface

Option A - Recommended first test from your Mac: Run port-forward locally on your Mac (with a working kubeconfig):

```
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

Open **https://localhost:8080**. The browser may warn about the initial self-signed certificate.

Option B - Run port-forward on the control-plane:

```
kubectl -n argocd port-forward svc/argocd-server \
  8080:443 --address=0.0.0.0
```

From the Mac open **https://192.168.178.49:8080**, provided firewall and LAN routing permit it. This binds the port on all host interfaces: use only on a trusted network, preferably with firewall restrictions. Do not expose it to the internet.

5. Obtain initial administrator password

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo
```

Username: **admin**. Password: the command output. After login, change the initial password and remove the initial secret once no longer needed.

6. Optional: permanent LAN access with NodePort

```
kubectl -n argocd patch svc argocd-server -p \
'{"spec":{"type":"NodePort"}}'
kubectl -n argocd get svc argocd-server
```

Find the assigned NodePort for port 443 and browse to https://192.168.178.49:<HTTPS_NODEPORT>. Ensure node firewall rules allow the chosen port. This is for LAN access; for a robust setup prefer an Ingress with TLS or MetalLB plus a LoadBalancer Service.

7. Troubleshooting

```
kubectl get events -n argocd --sort-by=.lastTimestamp
kubectl describe pod -n argocd <pod-name>
kubectl logs -n argocd deployment/argocd-server
kubectl get endpoints -n argocd argocd-server
```

ImagePullBackOff: check image registry connectivity and DNS. Pending: check node readiness, scheduling and resources. UI unreachable: check port-forward terminal, firewall and Service endpoints. For DNS diagnostics, inspect CoreDNS logs and Service resolution.

8. Next step: GitOps deployment

Once the UI works, connect your Git repository, create an Argo CD Application with the repository URL, path, destination cluster and namespace, and test a sync. Avoid storing GitHub tokens directly in manifests or Git.

Official documentation

https://argo-cd.readthedocs.io/en/stable/getting_started/

<https://argo-cd.readthedocs.io/en/stable/operator-manual/installation/>